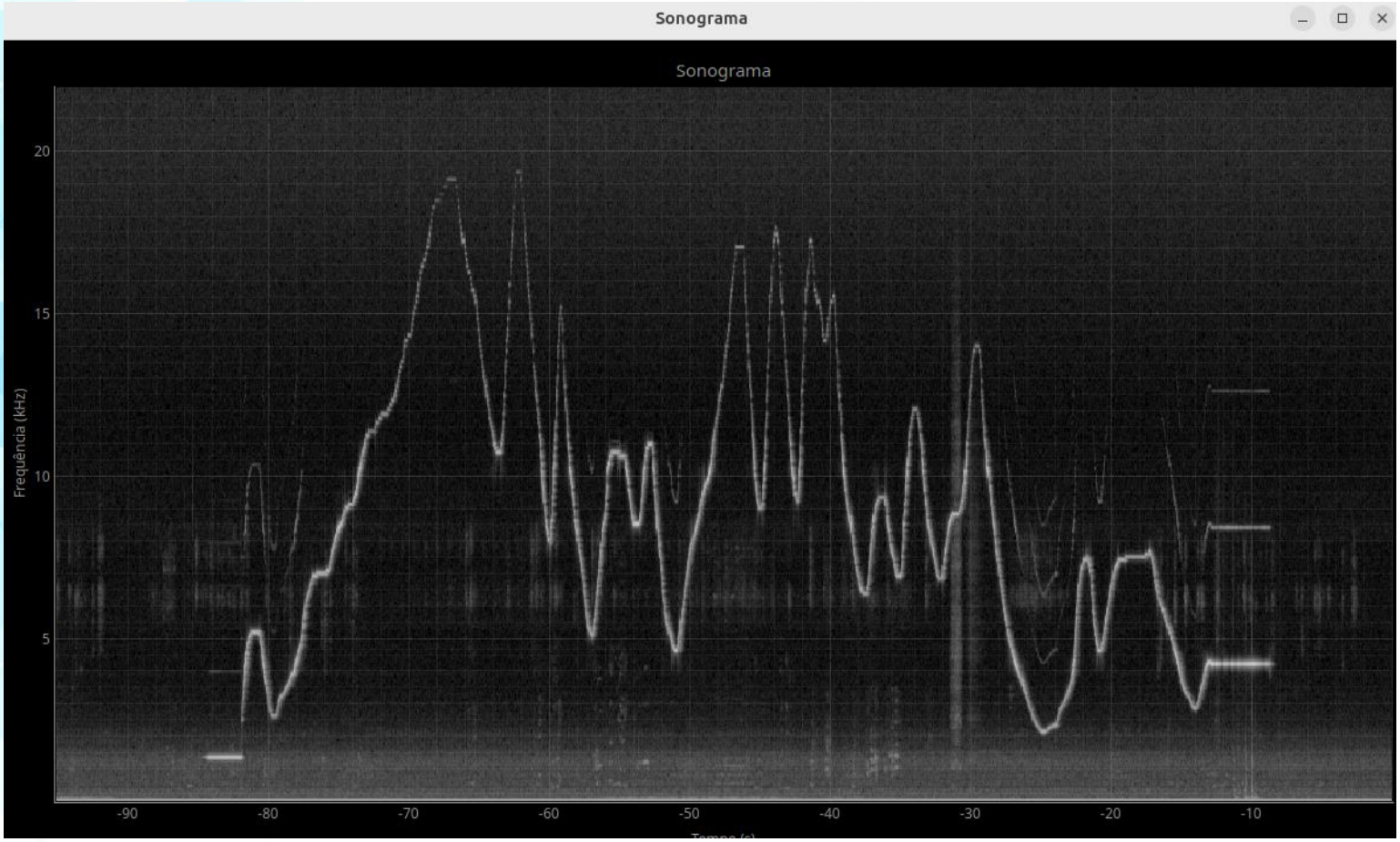
Desenvolvimento de aplicações para aquisição e análise de dados em tempo real com a linguagem Python



Por que adquirir/analisar sinais em tempo real ?

- Relacionar alterações nos sinais às atividades e aos eventos da fonte
- Detectar artefatos, ruídos e falhas ainda durante o ensaio
- Modificar parâmetros e validar resultados sem esperar resultado posterior

Exemplos na Engenharia Biomédica

- Aquisição de sinais de ECG/EMG/EEG
- Potenciais evocados
- Estudos em biomecânica (sensores inerciais)
- Sinais Respiratórios (Fluxo, pressão, oximetria)

Tarefas básicas de um sistema de aquisição/análise de sinais em tempo real



Por que Python?

- Amplo uso no ambiente acadêmico
- Desenvolvimento de aplicações é simples e rápido
- Grande disponibilidade de bibliotecas para processamento de sinais
- A mesma biblioteca pode ser usada no estudo offline e na aplicação
- Grande disponibilidade de drivers para sensores de sinais e atuadores eletromecânicos
- Razoável independência de plataforma

Por que Python não é tão frequentemente utilizado para esse tipo de aplicação?

- Tradicionalmente usado para análise de sinais
- Pouca cultura sobre aquisição de sinais em Python na maioria dos laboratórios
- Uso universal do MatplotLib para visualização (bonito, mas lento)
- Ideia (equivocada) de que linguagens interpretadas são incompatíveis com aplicações de tempo real – principalmente para demandas elevadas

Como funciona o interpretador do Python

Na verdade, na primeira execução, o programa em Python é COMPILADO em um bytecode pelo programa CPython

E o que é bytecode:

- **É uma "linguagem" intermediária que acelera a atuação do interpretador**
- **Pode ser lida em diferentes plataformas**

Infelizmente, no Python, a segunda etapa ainda é interpretada.

Atualmente, linguagens como Java e Matlab, utilizam um compilador Just In Time (JIT), que permite uma execução mais rápida do bytecode.

Em resumo: O Python é, de fato, LENTO.

MAS: as bibliotecas que ele chama (NumPy, SciPy, Scikit-Learn, ...) foram desenvolvidas em linguagens compiladas (C, Fortran, ...) e são muito rápidas !!!!

Estratégia para gerar código de elevado desempenho em Python

- **Usar o Python, o máximo possível, como linguagem de Script**
- **Deixar as tarefas de Aquisição/Processamento/Gravação/Exibição serem processadas por bibliotecas de alto desempenho**
- **Para exibição gráfica dinâmica e com elevada taxa de atualização de tela, existem bibliotecas muito mais eficientes que a Matplotlib. Uma delas é a PyQtGraph, que acessa diretamente a GPU da placa gráfica via OpenGL.**
- **A PyQtGraph é uma biblioteca montada em cima do PyQt e traz, com ela, ferramentas adicionais dessa biblioteca, como recursos de temporização (QTimer).**

O conceito de Tempo Real

- Garantia de execução dentro de uma janela temporal definida
- Hardware e software
- Não tem necessariamente a ver com velocidade
- Tem muito a ver com:
  - Agendamento rigoroso de processos
  - Atribuição rigorosa de prioridade de processo
  - Kernel preemptivo
  - Kernel reentrante
- Sistemas de tempo real rígido (hard realtime)
  - Determinísticos
- Sistemas de tempo real moderado (soft realtime)
  - Estocásticos

O conceito de Tempo Real (continuação)

- **Exemplo:**
  Considere um sistema onde um nó faça aquisição de sinais de ECG e transmita os sinais adquiridos em pacotes de dados a cada 100ms.

  Então, o nó responsável pelo processamento e visualização de sinais precisa GARANTIDAMENTE executar seu código em menos de 100ms, ou pacotes de dados serão perdidos.

  Se essa perda for inaceitável, o segundo nó deverá ser do tipo hard realtime.

  Se for aceitável, digamos, até 0,1% de perda, o segundo nó poderá ser do tipo soft realtime.
- **Aplicações que rodam em Windows, ou mesmo no Linux (convencional) são, na melhor das hipóteses, soft realtime.**
- **Com uma linguagem interpretada, como o Python, o que se pode fazer é ser rápido o suficiente para garantir um tempo de processamento bem abaixo do necessário e criar "observadores" para detectar perda de dados.**
- **Na maioria dos casos práticos de aquisição/análise, isso é suficiente!**

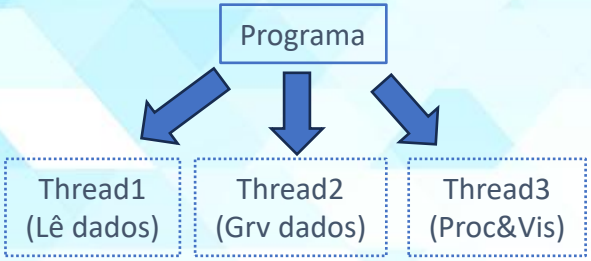
MultiThread

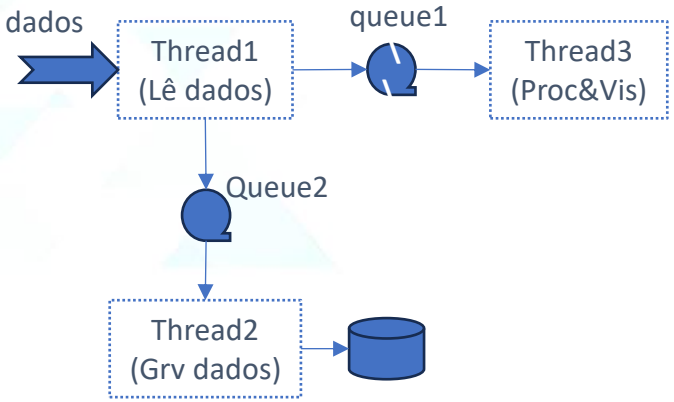
Como é possível um software "simultaneamente" ler os dados enviados por um sensor, gravar os dados recebidos e atualizar a interface gráfica, em taxas diferentes.

Isso é possível através de Multithreading, onde o programa é dividido em vários "programas" menores, chamados threads, que são executados "simultaneamente".

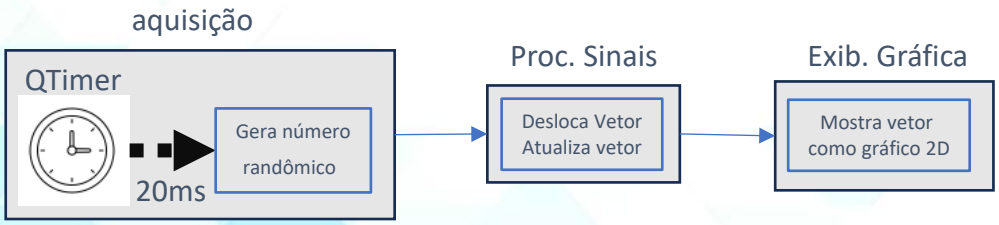
As threads normalmente executam de forma independente e trocam dados entre si por mecanismos de comunicação Interprocessos. Um desses mecanismos é a fila de dados Queue.

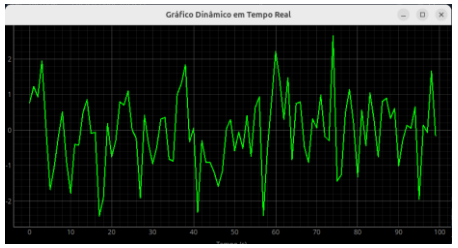
A queue é uma estrutura de dados que enche quando é escrita e esvazia quando é lida, sendo que os dados são lidos na mesma sequência em que foram escritos.



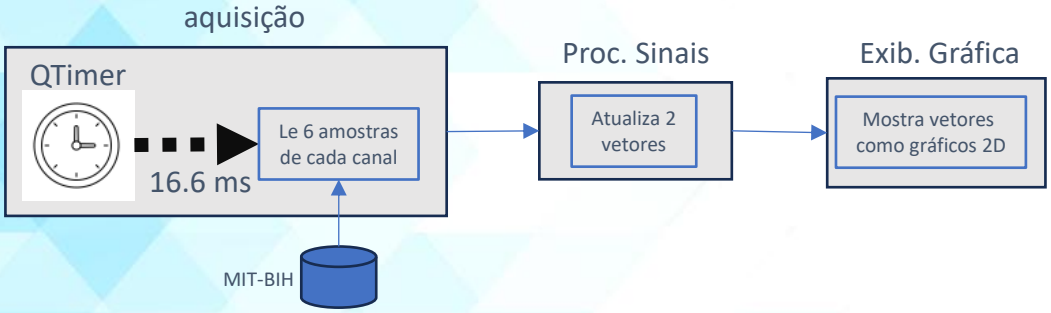


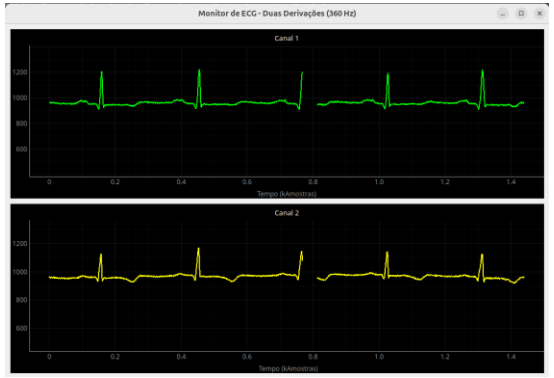
Exemplo 1: dyna1.py



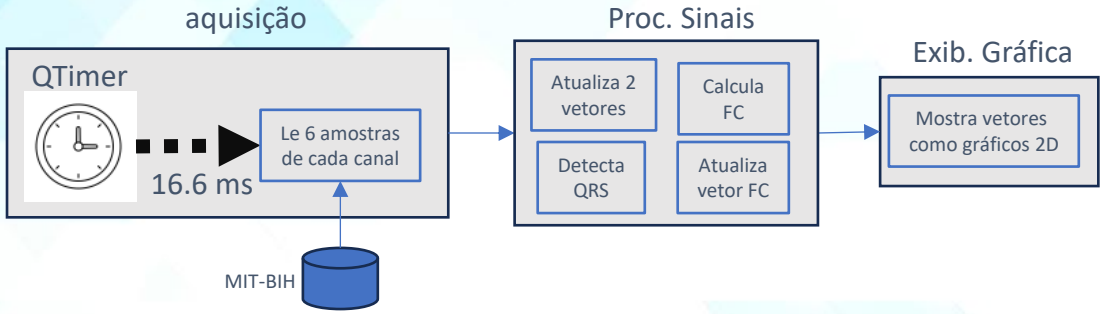


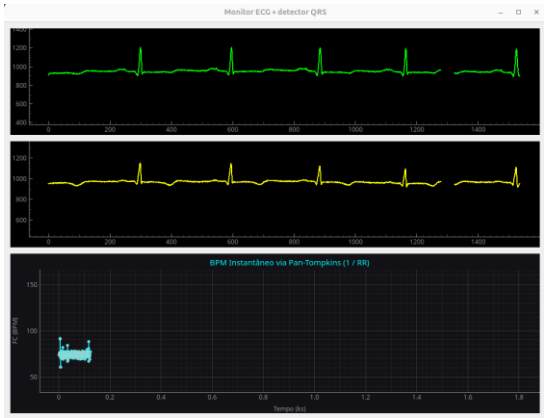
Exemplo 2: ecg1.py



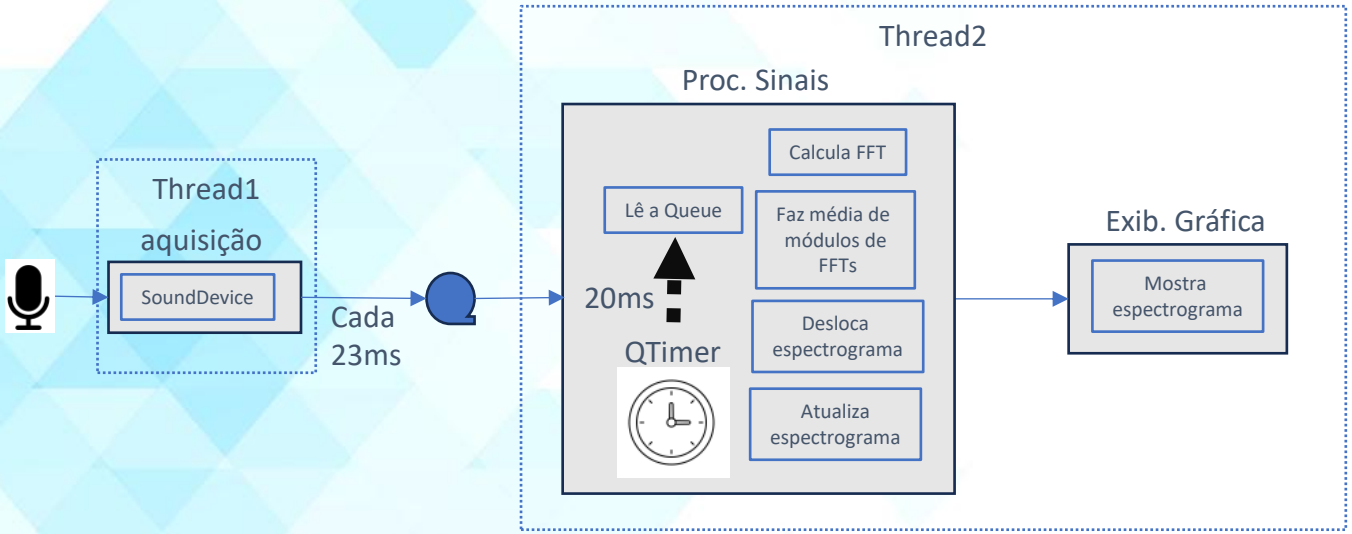


Exemplo 3: ecg2.py





Exemplo 4: sonograma_mic.py



Executando código em C++ no Python com pybind11

O pybind11 é uma biblioteca que permite transformar funções e classes C++ em módulos importáveis pelo Python.

- **O código C++ é compilado como uma biblioteca compartilhada (.so no Linux).**
- **O pybind11 cria a interface entre o código C++ e o interpretador Python.**
- **Funções e classes C++ passam a ser acessíveis por meio do comando import.**
- **A conversão de tipos de dados entre Python e C++ é realizada automaticamente para os tipos suportados.**



Executando código em C++ no Python com pybind11 (continuação)

exemplo.cpp

```
#include <pybind11/pybind11.h>

int soma(int a, int b) {
    return a + b;
}

PYBIND11_MODULE(exemplo, m) {
    m.def("soma", &soma);
}
```

```
c++ -O3 -Wall -shared -std=c++17 -fPIC $(python3 -m pybind11 --
includes) exemplo.cpp -o exemplo$(python3-config --extension-
suffix)
```



exemplo.cpython-312-x86_64-linux-gnu.so



```
import exemplo

resultado = exemplo.soma(10, 20)
print(resultado)
```

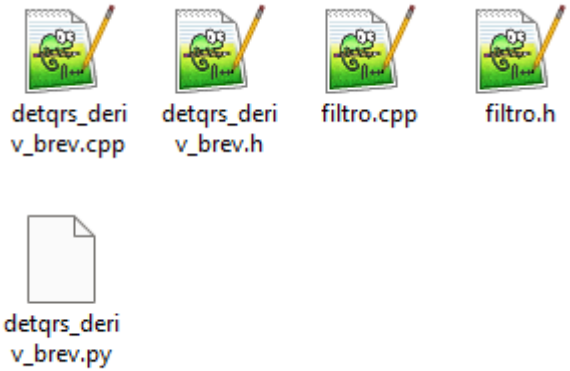
Aplicando no detetor de QRS deriv_brev

- Função desenvolvida em C++ e em Python para detectar complexos QRS.
- A versão em C++ permite operação em lote (todo o arquivo) e em blocos consecutivos do sinal (aplicações em tempo real).
- A versão em Python atualmente opera apenas em bloco.

detqrs_deriv_brev.h

```
#include <math.h>
#include "filtro.h"

class DetQRS
{
public:
	DetQRS(int dimSinal, int fAmost);
	~DetQRS();
	void Inicia(float *sinalPtr, int npts);
	int RetDimBlocoAmostras();
	int Processa(float *sinalPtr, int *qrs);
private:
	...
};
```



Aplicando no detetor de QRS deriv_brev (continuação)

detqrs_pybind.cpp

```
#include <pybind11/pybind11.h>
#include <pybind11/numpy.h>

#include "detqrs_deriv_brev.h"

namespace py = pybind11;

PYBIND11_MODULE(detqrs, m)
{
    m.doc() =
        "Binding Python do detector de QRS por derivacao e busca reversa";

    py::class_<DetQRS>(m, "DetQRS")
        .def(py::init<int, int>())
        .def("Inicia",[](DetQRS &self,
            py::array_t<float,py::array::c_style | py::array::forcecast> sinal)
            {
                py::buffer_info buf = sinal.request();
                self.Inicia(static_cast<float *>(buf.ptr),static_cast<int>(buf.size));
            }
        )
        .def("RetDimBlocoAmostras",&DetQRS::RetDimBlocoAmostras)
        .def("Processa",[](DetQRS &self,
                ...
                return py::make_tuple(n_qrs, saida);
            }
        );
}
```

```
c++ -O3 -Wall -shared -std=c++17 -fPIC \
$(python3 -m pybind11 --includes) \
 detqrs_pybind.cpp detqrs_deriv_brev.cpp filtro.cpp \
-o detqrs$(python3-config --extension-suffix)
```



detqrs.cpython-312-x86_64-linux-gnu.so

Aplicando no detetor de QRS deriv_brev (continuação)

O programa contabiliza.py aplica o detector deriv_brev em todos os 48 arquivos de MIT_BIH Arrhythmia Database (versão Fácil) e calcula o tempo total de processamento.

Para comparação de desempenho, o programa será executado com as duas versões do detector deriv_brev em Python, a original e a convertida de C++ via pybind11.

contabiliza.py + detqrs_deriv_brev.py

```
import numpy as np
from scipy.io import wavfile
import sys
import time

sys.path.append('../')

import detqrs_deriv_brev as detqrs

inicio = time.perf_counter()

nomeArqSaida="resultados_deriv_brev.txt"

# indique aqui se deseja, ou não, excluir o paciente 206
ignoraPac207=True

pastaSinais="../../MIT-BIH-FACIL/sinais/"
pastaAnotacoes="../../MIT-BIH-FACIL/anotações_mod/"
```

Tempo de processamento: 10.8s

contabiliza.py + detqrs.cpython-312-x86_64-linux-gnu.so

```
import numpy as np
from scipy.io import wavfile
import sys
import time

sys.path.append('../')

import detqrs

inicio = time.perf_counter()

nomeArqSaida="resultados_deriv_brev_lote.txt"

# indique aqui se deseja, ou não, excluir o paciente 206
ignoraPac207=True

pastaSinais="../../MIT-BIH-FACIL/sinais/"
pastaAnotacoes="../../MIT-BIH-FACIL/anotações_mod/"
```

Tempo de processamento: 1.4s

Processador i7-14000
(Linux ubuntu 24)

O programa ecg3.py refaz o ecg2.py utilizando o detector gerado via pinbind11.

Conclusão

Muitas atividades de Aquisição/Processamento/Gravação/Exibição de sinais em laboratório podem ser facilmente realizadas utilizando programação em Python.

Abordagem recomendada:

- **Utilizar a biblioteca PyQtGraph para sincronização e visualização de sinais (substituindo a MatPlotLib).**
- **Desacoplar a Aquisição das demais tarefas via Queues.**
- **Quando necessário, desenvolver em C/C++ e migrar para biblioteca Python usando pybind11.**

Esta apresentação e programas descritos podem ser obtidos em

https://github.com/marcelobossan-bcc/PythonTR.git